The `document` method expects the `flight` object; did you notice we didn't transform the `flight` Java object to JSON? That's because we've delegated the responsibility of marshaling and unmarshaling to the `JacksonJasonpMapper` class that we associated with the transport object earlier.

The `elasticsearchClient.index()` method takes an `IndexRequest` object and spits out `IndexResponse` (this aligns with what we've discussed about requests and responses).

C.8 Searching

Searching data using queries with the Java API client follows a similar path: invoking the `search()` method in the `ElasticsearchClient` class by passing the required query. There's one subtle difference, though—the other features expose a client per namespace, but the search feature does not. Let's see this in action.

Suppose we wish to search for a route from London to New York. We create a `match` query, providing "London New York" as the search criterion against the `route` field in a nicely formed DSL when working with Kibana.

Listing C.11 Searching for a route using a `match` query

```
GET flights/_search
{
  "query": {
    "match": {
      "route": "London New York"
    }
  }
}
```

This simple `match` query checks for a route with the keywords "London New York". We may get one or two results depending on how many records we've indexed. This query can also be written in Java using the Java API client.

Listing C.12 Searching using the Java API client

```
this.elasticsearchClient.search(searchRequest -> searchRequest
    .index(indexName)
    .query(queryBuilder ->
            queryBuilder.match(matchQBuilder->
                    matchQBuilder.field("route")
                            .query(searchText)))
        ,Flight.class
);
```

The `search()` method expects a search request, which is provided as a lambda expression to the method. The `query` is written using another lambda function—given a query request (`Query.Builder` object), we invoke the `match` function with the `MatchQuery.Builder` object. The JSON is converted to a `Flight` Java object, which is why `Flight.class` is provided as the argument to the `query` method.